📖 Αναλυτική Τεκμηρίωση Κώδικα — VLM_v2 Gateway Server

> **Έκδοση:** 2.0.0 | **Ημερομηνία:** Μάρτιος 2026 Αυτό το έγγραφο εξηγεί **γραμμή-γραμμή** κάθε αρχείο του server, πώς συνδέονται μεταξύ τους και πώς λειτουργεί το σύστημα στο σύνολό του — χωρίς προϋπόθεση γνώσης Python ή προγραμματισμού.

🗺️ Χάρτης Αρχείων

```
VLM_v2/
└── v2/
    └── app/
        ├── main.py                        ← Σημείο εκκίνησης
        ├── config.py                      ← Κεντρικές ρυθμίσεις
        ├── state.py                       ← Κοινή μνήμη (RAM) του server
        ├── receivers/
        │   └── gateway_server.py          ← Δημιουργία FastAPI app
        ├── api/
        │   ├── api_router.py              ← Κεντρικός δρομολογητής URL
        │   └── routers/
        │       ├── core.py                ← Health check, Pi status
        │       ├── mobile.py              ← Λήψη εικόνας, περιγραφή, long-
polling
        │       └── tools.py               ← Βοηθητικά endpoints (VLM/YOLO
on-demand)
        ├── pipelines/
        │   ├── describe.py                ← Pipeline: εικόνα → περιγραφή
κειμένου
        │   └── detect.py                  ← Pipeline: εικόνα → ανίχνευση
αντικειμένων
        ├── models/
        │   ├── Qwen/
        │   │   └── qwen3_vl_lmstudio.py   ← Επικοινωνία με LM Studio /
Qwen
        │   ├── yolo_model.py              ← YOLO: ανίχνευση & χρώμα
φαναριού
        │   └── weights/                   ← Αρχεία βαρών (yolo26n.pt)
        └── utils/
            └── image_preprocess.py        ← Βελτίωση ποιότητας εικόνας
```

🔌 Αρχιτεκτονική: Πώς Επικοινωνούν Όλοι

```
┌──────────────┐   POST /mobile/process   ┌──────────────────────────┐
│ Raspberry Pi │ ────────────────────────► │                          │
│  (τα γυαλιά) │                           │   Gateway Server         │
```

```
└──────────────┘                              │   (FastAPI, port 5050)   │
                                              │                          │
┌──────────────┐   GET /mobile/caption_       │  ┌──────────────────┐    │
│ Flutter App  │ ◄──────────────────────────  │  │  state.py (RAM)  │    │
│  (κινητό)    │   next (long-polling)        │  └──────────────────┘    │
└──────────────┘                              │          │               │
                                              │          ▼               │
                                              │   LM Studio (Qwen)       │
                                              │   (port 9094, local)     │
                                              └──────────────────────────┘
```

Με απλά λόγια:

1. Το **Raspberry Pi** (τα γυαλιά) τραβά φωτογραφία και την ανεβάζει στον server.
2. Ο server τη στέλνει στο **LM Studio** (ένα πρόγραμμα που τρέχει τοπικά το μοντέλο Qwen) για να πάρει περιγραφή.
3. Η περιγραφή αποθηκεύεται στη **μνήμη RAM** του server.
4. Η **Flutter εφαρμογή** στο κινητό ρωτά συνεχώς "υπάρχει νέα περιγραφή;" και μόλις ναι, την εμφανίζει.

📄 `main.py` — Το Σημείο Εκκίνησης

```
"""
Main entry point for the VLM_v2 system.
Starts the Gateway Server (FastAPI).
"""
```

> Αυτό το σχόλιο (documentation string) απλώς λέει ότι αυτό είναι το αρχείο που ξεκινά τα πάντα.

```
from __future__ import annotations
```

> Αυτή η γραμμή λέει στην Python να χρησιμοποιεί ένα πιο σύγχρονο σύστημα τύπων (type hints). Δεν επηρεάζει τη λειτουργία.

```
import sys
import os
from pathlib import Path
```

> **Εισαγωγή βιβλιοθηκών:**
>
> - `sys` → για διαχείριση της Python διαδρομής (path)
> - `os` → για αλληλεπίδραση με το λειτουργικό σύστημα

> - `Path` → για χειρισμό διαδρομών αρχείων/φακέλων με έξυπνο τρόπο

```
# Ensure the project root is in the python path
project_root = Path(__file__).resolve().parent.parent.parent
if str(project_root) not in sys.path:
    sys.path.insert(0, str(project_root))
```

> **Αυτό λύνει ένα πολύ σημαντικό πρόβλημα:**
>
> - `Path(__file__)` → "πού βρίσκεται αυτό το αρχείο;"
> - `.resolve()` → πάρε την απόλυτη διαδρομή (π.χ. `/Users/dimi/.../main.py`)
> - `.parent.parent.parent` → ανέβα τρία επίπεδα πάνω: από `app/` → `v2/` → `VLM_v2/`
> - `sys.path.insert(0, ...)` → λέει στην Python "κοίτα πρώτα σε αυτόν τον φάκελο για να βρεις modules"
>
> **Γιατί χρειάζεται;** Χωρίς αυτό, όταν γράφουμε `from v2.app.config import ...` αλλού, η Python δεν θα ξέρει πού να ψάξει.

```
from v2.app.receivers.gateway_server import run_server
```

> **Εισαγωγή της κύριας λειτουργίας:** Από το αρχείο `receivers/gateway_server.py` παίρνουμε τη συνάρτηση `run_server`. Αυτή ξεκινά τον FastAPI server.

```
def main():
    print("="*50)
    print("ImageDescribeLLM V2 - System Starting")
    print("="*50)
```

> Ορισμός της κύριας συνάρτησης `main()`. Εκτυπώνει ένα banner εκκίνησης στο τερματικό (50 ίσα "=" + μήνυμα).

```
    try:
        # Start the FastAPI server
        run_server(host="0.0.0.0", port=5050, ssl=False)
```

> Καλεί τη `run_server` με παραμέτρους:
>
> - `host="0.0.0.0"` → άκουγε σε **όλες** τις δικτυακές διεπαφές (όχι μόνο localhost), ώστε να φτάνει το Pi και το κινητό
> - `port=5050` → χρησιμοποίησε την πόρτα 5050

> - `ssl=False` → χωρίς κρυπτογράφηση HTTPS (HTTP απλό)
>
> 🔗 **Σύνδεση:** Πηγαίνει στο `gateway_server.py` → `run_server()`

```
    except KeyboardInterrupt:
        print("\nSystem stopped by user.")
    except Exception as e:
        print(f"\nA critical error occurred: {e}")
        import traceback
        traceback.print_exc()
    finally:
        print("System shutdown complete.")
```

> **Διαχείριση σφαλμάτων:**
>
> - `KeyboardInterrupt` → αν ο χρήστης πατήσει `Ctrl+C`, τυπώνει "σταμάτησε ο χρήστης"
> - `Exception as e` → οποιοδήποτε άλλο σφάλμα → το εκτυπώνει μαζί με stack trace (ίχνος σφάλματος)
> - `finally` → **πάντα** εκτελείται, ακόμα και αν υπάρχει σφάλμα → μήνυμα "shutdown"

```
if __name__ == "__main__":
    main()
```

> **Αυτό είναι ο "διακόπτης εκκίνησης":** Στην Python, κάθε αρχείο έχει μια "ταυτότητα" που λέγεται `__name__`. Όταν τρέχεις ένα αρχείο **απευθείας** (π.χ. `python main.py`), το `__name__` γίνεται `"__main__"`. Αν το αρχείο **εισαχθεί από αλλού** (`import`), δεν τρέχει αυτός ο κώδικας.
>
> **Αποτέλεσμα:** Μόνο όταν τρέχεις `python main.py` απευθείας ξεκινά η `main()`.

🔗 Συνδέσεις του `main.py`

Συνδέεται με	Μέσω
`receivers/gateway_server.py`	`from v2.app.receivers.gateway_server import run_server`

⚙️ `config.py` — Κεντρικές Ρυθμίσεις

> Αυτό το αρχείο είναι ο "πίνακας ελέγχου" του συστήματος. **Όλοι** οι άλλοι κώδικες έρχονται εδώ για να πάρουν ρυθμίσεις. Αλλάζοντας κάτι εδώ, αλλάζει η συμπεριφορά ολόκληρου του συστήματος.

```
import os
from pathlib import Path
```

```
import torch
```

> - `os` → διαβάζει μεταβλητές περιβάλλοντος (environment variables)
> - `Path` → χειρισμός διαδρομών
> - `torch` → βιβλιοθήκη deep learning (PyTorch) — χρησιμοποιείται μόνο για να ανιχνευθεί ο τύπος υλικού (GPU/CPU)

```
os.environ.setdefault("HF_HUB_DISABLE_PROGRESS_BARS", "1")
os.environ.setdefault("HF_HUB_DISABLE_TELEMETRY", "1")
os.environ.setdefault("TRANSFORMERS_NO_ADVISORY_WARNINGS", "1")
os.environ.setdefault("TQDM_DISABLE", "1")
```

> **"Σιωπηλός τρόπος λειτουργίας" για βιβλιοθήκες AI:**
>
> - Απενεργοποιεί τις μπάρες προόδου του Hugging Face
> - Απενεργοποιεί την αποστολή τηλεμετρίας (ανώνυμα στατιστικά χρήσης)
> - Κρύβει προειδοποιητικά μηνύματα των Transformers
> - Απενεργοποιεί το `tqdm` (βιβλιοθήκη μπαρών προόδου)
>
> **Αποτέλεσμα:** Πιο "καθαρή" εκτύπωση στο τερματικό κατά τη λειτουργία.

```
APP_ROOT = Path(__file__).resolve().parent       # .../v2/app/
V2_ROOT = APP_ROOT.parent                        # .../v2/
PROJECT_ROOT = V2_ROOT.parent                    # .../VLM_v2/
```

> **Διαδρομές βάσης:**
>
> - `APP_ROOT` = ο φάκελος `app/`
> - `V2_ROOT` = ο φάκελος `v2/`
> - `PROJECT_ROOT` = ο κορμός του project, `VLM_v2/`
>
> Όλοι οι άλλοι φάκελοι χτίζονται πάνω σε αυτές τις διαδρομές.

```
DATA_DIR = Path(os.environ.get("DATA_DIR", PROJECT_ROOT / "Data"))
DATA_DIR.mkdir(parents=True, exist_ok=True)
```

> - `os.environ.get("DATA_DIR", ...)` → "διάβασε την μεταβλητή `DATA_DIR` από το σύστημα. Αν δεν υπάρχει, χρησιμοποίησε `VLM_v2/Data/`"
> - `.mkdir(parents=True, exist_ok=True)` → δημιούργησε τον φάκελο αν δεν υπάρχει (χωρίς σφάλμα αν υπάρχει ήδη)
>
> **Αποτέλεσμα:** Εγγυάται ότι ο φάκελος `Data/` υπάρχει πάντα.

```
def get_device() -> torch.device:
    if torch.cuda.is_available():
        return torch.device("cuda")
    if hasattr(torch.backends, "mps") and
torch.backends.mps.is_available():
        return torch.device("mps")
    return torch.device("cpu")
```

> **Αυτόματη επιλογή υλικού για AI:**
>
> 1. Αν υπάρχει κάρτα γραφικών NVIDIA (CUDA) → χρησιμοποίησέ την (πιο γρήγορο)
> 2. Αν είμαστε σε Mac με Apple Silicon (MPS) → χρησιμοποίησε τον chip (δεύτερη επιλογή)
> 3. Αλλιώς → CPU (πιο αργό αλλά δουλεύει παντού)
>
> **Σημαντικό:** Χρησιμοποιείται από τα μοντέλα BLIP/LLaVa. Το **Qwen μέσω LM Studio** δεν χρειάζεται αυτή τη λογική γιατί τρέχει ανεξάρτητα.

```
def get_dtype() -> torch.dtype:
    device = get_device()
    if device.type in ("cuda", "mps"):
        return torch.float16
    return torch.float32
```

> - `float16` = μισής ακρίβειας αριθμοί (καταλαμβάνουν λιγότερη μνήμη στο GPU)
> - `float32` = κανονικής ακρίβειας (για CPU που δεν έχει μνήμη VRAM)

```
def to_rel_path(path: str | Path, include_leading_slash: bool = True) ->
str:
    ...
```

> Βοηθητική συνάρτηση που μετατρέπει μακριές διαδρομές σε πιο σύντομες για το log. Π.χ. `/Users/dimi/.../Data/image.jpg` → `/Data/image.jpg`.

```
YOLO_WEIGHTS_DIR = APP_ROOT / "models" / "weights"
YOLO_WEIGHTS = Path(os.environ.get("YOLO_WEIGHTS", YOLO_WEIGHTS_DIR /
"yolo26n.pt"))
YOLO_WEIGHTS.parent.mkdir(parents=True, exist_ok=True)
```

> - Καθορίζει πού βρίσκονται τα "βάρη" (weights) του μοντέλου YOLO (το αρχείο `.pt` που περιέχει το εκπαιδευμένο μοντέλο)

> - Δημιουργεί τον φάκελο `weights/` αν δεν υπάρχει

```
PIPELINE_NAME = "describe_loop_return"
MODEL = "qwen3_vl"
PREPROC = "opencv"
```

> **Επιλογές pipeline:**
>
> - `PIPELINE_NAME` → ποια ροή λειτουργίας να χρησιμοποιηθεί (η κύρια: "describe_loop_return")
> - `MODEL` → ποιο μοντέλο AI να χρησιμοποιηθεί. Επιλογές: `"qwen3_vl"`, `"llava"`, `"blip2"`, `"blip_large"`
> - `PREPROC` → πώς να προεπεξεργαστούν οι εικόνες: `"opencv"` ή `"pillow"`

```
LM_STUDIO_BASE_URL = os.environ.get("LM_STUDIO_BASE_URL",
"http://127.0.0.1:9094")
LM_STUDIO_MODEL_NAME = os.environ.get("LM_STUDIO_MODEL_NAME", "qwen3-vl-
32b-instruct")
```

> - Διεύθυνση του LM Studio (τρέχει τοπικά στον ίδιο υπολογιστή, port 9094)
> - Όνομα του μοντέλου που είναι φορτωμένο στο LM Studio
> - Και τα δύο μπορούν να αλλαχθούν μέσω environment variables χωρίς αλλαγή κώδικα

```
TIME_LOGS = os.environ.get("TIME_LOGS", "ON").strip().upper() == "ON"
YOLO_ENABLED = os.environ.get("YOLO_ENABLED", "OFF").strip().upper() ==
"ON"
```

> - `TIME_LOGS` → αν `"ON"`, καταγράφει χρόνους (latency) σε αρχείο CSV. Default: ON.
> - `YOLO_ENABLED` → αν `"ON"`, ενεργοποιεί τη χρήση YOLO μαζί με τη VLM περιγραφή. Default: OFF.
>
> 🔗 **Χρήση:** Το `mobile.py` ελέγχει `TIME_LOGS` και `YOLO_ENABLED` σε κάθε αίτηση.

🔗 Ποιοι κώδικες εισάγουν το `config.py`

Αρχείο	Τι εισάγει
`mobile.py`	`DATA_DIR`, `TIME_LOGS`, `YOLO_ENABLED`
`tools.py`	`DATA_DIR`
`core.py`	— (έμμεσα μέσω state)

Αρχείο	Τι εισάγει
`describe.py`	`MODEL`, `PREPROC`, `DATA_DIR`, `get_device`, `get_dtype`
`detect.py`	`YOLO_ENABLED`
`qwen3_vl_lmstudio.py`	`LM_STUDIO_BASE_URL`, `LM_STUDIO_MODEL_NAME`
`yolo_model.py`	`YOLO_WEIGHTS`

🧠 `state.py` — Η Κοινή Μνήμη (RAM) του Server

> Αυτό το αρχείο ορίζει **μεταβλητές που ζουν στη RAM** κατά τη διάρκεια λειτουργίας του server. Δεν αποθηκεύονται στο δίσκο — αν ο server σταματήσει, χάνονται. Είναι σαν ένα "whiteboard" που όλα τα μέρη του server βλέπουν και ενημερώνουν.

```
from typing import Any, Dict, Optional
import asyncio
```

> - `typing` → για να ορίσουμε τους τύπους των μεταβλητών (καλή πρακτική)
> - `asyncio` → η βιβλιοθήκη για ασύγχρονο προγραμματισμό (απαραίτητη για long-polling)

```
SOURCES = ("pi", "mobile")
```

> Ορίζει τις δύο πηγές από τις οποίες μπορεί να έρθει εικόνα: το Pi (τα γυαλιά) ή το κινητό.

```
latest_by_source: Dict[str, Optional[Dict[str, Any]]] = {s: None for s in
SOURCES}
```

> **Η πιο σημαντική μεταβλητή:** Ένα λεξικό που κρατά **την τελευταία περιγραφή για κάθε πηγή**.
>
> Μοιάζει με:
>
> ```
> {
>  "pi":     {"caption": "Ένας άνδρας...", "created_at": 1741300000,
> "source": "pi"},
>  "mobile": None   # αν δεν έχει έρθει ακόμα εικόνα από κινητό
> }
> ```
>
> - `Dict[str, ...]` → λεξικό με κλειδί string
> - `Optional[Dict...]` → η τιμή μπορεί να είναι λεξικό ή `None`

> - `{s: None for s in SOURCES}` → αρχικά όλα `None` (δεν έχει έρθει τίποτα)

```
latest_pi: Optional[Dict[str, Any]] = None
```

> Παλιά μεταβλητή που κρατά την τελευταία περιγραφή από το Pi. Διατηρείται για **backward compatibility** (παλιότερος κώδικας μπορεί να τη χρησιμοποιεί).

```
last_caption_ts: Dict[str, int] = {s: 0 for s in SOURCES}
```

> Timestamp (Unix epoch, δευτερόλεπτα από 1/1/1970) της τελευταίας περιγραφής ανά πηγή. Αρχικά `0` (δεν έχει έρθει τίποτα). Χρησιμοποιείται για να ξέρει το long-polling αν υπάρχει νέο δεδομένο.

```
caption_cond: Dict[str, asyncio.Condition] = {}
```

> **"Ασύγχρονες συνθήκες" ανά πηγή** — ο μηχανισμός που επιτρέπει το long-polling.
>
> Φανταστείτε το ως "καμπανάκι": ο server **κρατά "κοιμισμένη"** τη Flutter εφαρμογή εδώ, και μόλις έρθει νέα περιγραφή, **χτυπά το καμπανάκι** και ξυπνά τη Flutter να πάρει τα δεδομένα.
>
> Ξεκινά ως κενό dictionary — γεμίζει στην εκκίνηση από τη `init_async_primitives()`.

```
last_seen_pi: int = 0
```

> Timestamp της τελευταίας φορά που το Pi έστειλε εικόνα. Χρησιμοποιείται για να ξέρουμε αν το Pi είναι "online" (αν πέρασαν >15 δευτερόλεπτα, θεωρείται offline).

```
status_cond: Optional[asyncio.Condition] = None
```

> Παρόμοιο "καμπανάκι" αλλά για την **κατάσταση του Pi** (online/offline). Όταν το Pi στείλει εικόνα, ξυπνά όποιον ρωτά "είναι online το Pi;".

```
def init_async_primitives() -> None:
    global caption_cond, status_cond
    if not caption_cond:
        caption_cond = {s: asyncio.Condition() for s in SOURCES}
```

```
    if status_cond is None:
        status_cond = asyncio.Condition()
```

> **Γιατί δεν δημιουργούμε τα `Condition` αμέσως;**
>
> Τα `asyncio.Condition` πρέπει να δημιουργηθούν **αφού** ξεκινήσει ο event loop της asyncio. Αν τα δημιουργήσουμε κατά την εισαγωγή του module (import time), μπορεί να δέσουν σε λάθος event loop και να υπάρξουν σφάλματα.
>
> Αυτή η συνάρτηση καλείται από τον `gateway_server.py` κατά την εκκίνηση.

🔗 Ποιοι κώδικες χρησιμοποιούν το `state.py`

Αρχείο	Τι κάνει
`mobile.py`	Γράφει `latest_by_source`, `last_caption_ts`, `last_seen_pi` — ξυπνά `caption_cond`, `status_cond`
`core.py`	Διαβάζει `last_seen_pi`, χρησιμοποιεί `status_cond` για long-polling
`gateway_server.py`	Καλεί `init_async_primitives()` στην εκκίνηση

🌐 `receivers/gateway_server.py` — Δημιουργία του FastAPI Server

> Αυτό το αρχείο "στήνει" τον web server. Ορίζει τι γίνεται κατά την εκκίνηση και πώς οι διευθύνσεις URL συνδέονται με κώδικα.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
import uvicorn
from v2.app.api.api_router import router
from v2.app import state
```

> - `FastAPI` → το web framework (σαν Django ή Flask, αλλά πιο σύγχρονο)
> - `CORSMiddleware` → επιτρέπει σε browsers/apps από άλλα domains να καλούν τον server
> - `uvicorn` → ο ASGI web server που "τρέχει" το FastAPI app
> - `router` → ο κεντρικός δρομολογητής URL από `api_router.py`

```
def create_app() -> FastAPI:
    app = FastAPI(title="ImageDescribeLLM Gateway", version="2.0.0")

    app.add_middleware(
        CORSMiddleware,
        allow_origins=["*"],
        allow_methods=["*"],
        allow_headers=["*"],
```

```
    )

    @app.on_event("startup")
    async def startup():
        state.init_async_primitives()

    app.include_router(router)
    return app
```

> 1. Δημιουργεί το FastAPI app με τίτλο και version
> 2. Προσθέτει CORS: `allow_origins=["*"]` = οποιοσδήποτε μπορεί να καλέσει τον server (χρήσιμο για το κινητό)
> 3. Ορίζει "startup hook": μόλις ξεκινήσει ο server, καλεί `state.init_async_primitives()` για να δημιουργήσει τα asyncio Conditions
> 4. Συνδέει όλα τα routes (URLs) μέσω του `router`

```
def run_server(host="0.0.0.0", port=5050, ssl=False):
    app = create_app()
    uvicorn.run(app, host=host, port=port, ...)
```

> Αυτή είναι η συνάρτηση που καλεί η `main.py`. Δημιουργεί και τρέχει τον server.

🔗 Συνδέσεις του `gateway_server.py`

Συνδέεται με	Γιατί
`main.py`	Καλείται από εκεί (`run_server`)
`api/api_router.py`	Εισάγει τον κεντρικό router
`state.py`	Αρχικοποιεί τα async primitives στην εκκίνηση

🗺️ `api/api_router.py` — Ο Κεντρικός Δρομολογητής

```
from fastapi import APIRouter
from v2.app.api.routers import core, mobile, tools

router = APIRouter()
router.include_router(core.router)
router.include_router(mobile.router, prefix="/mobile")
router.include_router(tools.router, prefix="/tools")
```

> Συγκεντρώνει τρεις επιμέρους router και τους ορίζει "βάσεις" URL:
>
> - Χωρίς prefix → `/`, `/health`, `/api/pi/status`, `/api/pi/status_next`

> - `/mobile` → `/mobile/process`, `/mobile/caption_latest`, `/mobile/caption_next`
> - `/tools` → `/tools/vlm`, `/tools/yolo`

📡 `api/routers/core.py` — Health Check & Pi Status

`GET /`

```
@router.get("/")
async def index():
    return {
        "service": "ImageDescribeLLM Gateway",
        "status": "active",
        "version": "2.0.0",
        ...
    }
```

> Απλό "ζωντανός είσαι;" endpoint. Επιστρέφει πληροφορίες για τον server.

`GET /health`

```
@router.get("/health")
async def health():
    return {"status": "ok"}
```

> Ο πιο απλός έλεγχος — επιστρέφει `{"status": "ok"}`. Χρήσιμο για monitoring.

`GET /api/pi/status`

```
@router.get("/api/pi/status")
async def pi_status():
    now = int(time.time())
    last_seen = int(state.last_seen_pi)
    online = (now - last_seen) <= 15 if last_seen > 0 else False
    return {"online": online, "last_seen": last_seen}
```

> Ελέγχει αν το Pi είναι online:
>
> - `time.time()` → η τρέχουσα ώρα σε Unix epoch (αριθμός δευτερολέπτων)
> - `state.last_seen_pi` → πότε είδαμε τελευταία φορά το Pi
> - `(now - last_seen) <= 15` → αν πέρασαν ≤15 δευτερόλεπτα → online
> - Επιστρέφει `{"online": true/false, "last_seen": timestamp}`

`GET /api/pi/status_next` — Long-Polling για Pi Status

```
@router.get("/api/pi/status_next")
async def pi_status_next(since: Optional[int] = ..., timeout: int = 15):
```

> Αυτό το endpoint χρησιμοποιεί **long-polling**. Η Flutter εφαρμογή το καλεί και ο server:
>
> - Αν υπάρχει νέα κατάσταση (last_seen > since) → απαντά αμέσως
> - Αλλιώς → **κρατά ανοιχτή** τη σύνδεση μέχρι να υπάρξει αλλαγή ή να λήξει το timeout (15 δευτ.)
>
> **Τεχνική:** Χρησιμοποιεί `asyncio.wait_for` + `state.status_cond` για να "κοιμηθεί" και να "ξυπνήσει" αποτελεσματικά.

📱 `api/routers/mobile.py` — Ο Κύριος Router (Pi → Server → State)

> Αυτός είναι ο **πιο σημαντικός** router. Εδώ γίνεται η λήψη εικόνων από το Pi, η επεξεργασία τους και η αποστολή περιγραφών στο κινητό.

`POST /mobile/process` — Λήψη Εικόνας από Pi

```
UPLOAD_FOLDER = Path(DATA_DIR) / "received_images"
UPLOAD_FOLDER.mkdir(parents=True, exist_ok=True)
FIXED_FILENAME = "current_image.jpg"
```

> - Ορίζει τον φάκελο αποθήκευσης: `Data/received_images/`
> - Δημιουργεί τον φάκελο αν δεν υπάρχει
> - Κάθε νέα εικόνα **αντικαθιστά** την προηγούμενη με το ίδιο όνομα `current_image.jpg`

```
@router.post("/process")
async def process_image(
    image: UploadFile = File(...),
    src: Optional[str] = Query(default=None, alias="src")
):
```

> - `UploadFile` → αρχείο που ανεβαίνει (multipart upload)
> - `src` → query parameter: από πού έρχεται η εικόνα; π.χ. `?src=pi`

Βήμα 1: Αποθήκευση εικόνας

```
    file_path = UPLOAD_FOLDER / FIXED_FILENAME
    with file_path.open("wb") as buffer:
```

```
        shutil.copyfileobj(image.file, buffer)
```

> Αποθηκεύει τη ληφθείσα εικόνα στο δίσκο ως `Data/received_images/current_image.jpg`.
> `"wb"` = write binary (γράψε δυαδικά, όχι κείμενο).

Βήμα 2: Εκτέλεση Pipeline Περιγραφής

```
    from v2.app.pipelines.describe import run as run_describe
    response = await run_in_threadpool(run_describe, str(file_path),
profile=profile)
```

> - Εισάγει τη συνάρτηση `run` από το `describe.py`
> - `run_in_threadpool` → τρέχει τη `run_describe` σε **ξεχωριστό νήμα** (thread pool)
> - **Γιατί;** Η κλήση στο LM Studio παίρνει χρόνο. Αν τρέχαμε στον κύριο thread, ο server θα "πάγωνε" και δεν θα μπορούσε να εξυπηρετήσει άλλες αιτήσεις. Με `await run_in_threadpool`, ο event loop παραμένει ελεύθερος.
>
> 🔗 **Σύνδεση:** Πηγαίνει στο `pipelines/describe.py` → `run()`

Βήμα 3: Καταγραφή Χρόνων (Time Logs)

```
    elapsed = perf_counter() - t0
    if TIME_LOGS:
        log_path = logs_dir / "timestamps.csv"
        line = f"{ts0} | {yolo_field} | {desc_sec:.3f} | {total_sec:.3f}\n"
        # ...εγγραφή στο CSV...
```

> Αν `TIME_LOGS = True` (από config.py), γράφει στο `Data/logs/timestamps.csv`:
>
> - Πότε λήφθηκε η εικόνα
> - Πόση ώρα πήρε το YOLO (αν ενεργό)
> - Πόση ώρα πήρε η περιγραφή
> - Συνολικός χρόνος

Βήμα 4: Ενημέρωση State

```
    caption_text = str(response)
    now_epoch = int(time.time())
    source = "pi" if src_low == "pi" else "mobile"

    payload = {"caption": caption_text, "created_at": now_epoch, "source":
source}
    state.latest_by_source[source] = payload
    state.last_caption_ts[source] = now_epoch
```

```
if source == "pi":
    state.latest_pi = payload
    state.last_seen_pi = now_epoch
    # Ξύπνα status long-pollers
    async with cond:
        cond.notify_all()
```

> - Αποθηκεύει την περιγραφή στη RAM (`state.latest_by_source`)
> - Ενημερώνει timestamp
> - Αν η πηγή είναι Pi, ενημερώνει και το `last_seen_pi` (Pi = online)
> - **Ξυπνά** όλους τους waiters του `status_cond` (η Flutter ξέρει ότι το Pi είναι alive)

Βήμα 5: Ξύπνα Caption Long-Pollers

```
cond = state.caption_cond.get(source)
async with cond:
    cond.notify_all()
```

> Ξυπνά τη Flutter εφαρμογή που περιμένει νέα περιγραφή μέσω `caption_next`.

Βήμα 6: Εγγραφή Αρτεφάκτων στο Δίσκο

```
latest_txt = gen_dir / "image_description.txt"
with latest_txt.open("w") as f:
    f.write(caption_text)

# JSON ιστορικό (3 τελευταίες)
items = ([payload] + items)[:3]
hist_json.write_text(json.dumps(items, ...))
```

> - Αποθηκεύει την τελευταία περιγραφή στο `Data/Generated_Text/image_description.txt`
> - Κρατά ιστορικό 3 τελευταίων περιγραφών στο `Data/Generated_Text/captions.json`
> - **Σημαντικό:** Αυτά είναι μόνο για αναφορά/debugging. Η Flutter **δεν** τα διαβάζει — χρησιμοποιεί τη RAM μέσω των API endpoints.

```
return {"description": response}
```

> Επιστρέφει JSON απάντηση στο Pi (200 OK): `{"description": "...κείμενο..."}`.

`GET /mobile/caption_latest` — Άμεση Λήψη Τελευταίας Περιγραφής

```
@router.get("/caption_latest")
async def caption_latest(since: Optional[int] = ..., source: Optional[str]
= "pi"):
```

> - Αν υπάρχει περιγραφή νεότερη από `since` → επιστρέφει `{"caption": ..., "created_at": ..., "source": ...}`
> - Αν όχι → `204 No Content`
>
> **Χρήση:** Άμεση (non-blocking) ερώτηση: "υπάρχει κάτι νέο;"

`GET /mobile/caption_next` — Long-Polling για Νέα Περιγραφή

```
@router.get("/caption_next")
async def caption_next(source="pi", since=None, timeout=25):
```

> **Αυτό είναι το κύριο endpoint που χρησιμοποιεί η Flutter εφαρμογή:**
>
> 1. Έλεγχος αμέσως: αν υπάρχει περιγραφή νεότερη από `since` → επιστρέφει αμέσως
> 2. Αν όχι → **κρατά ανοιχτή** τη σύνδεση μέχρι:
> - Να έρθει νέα περιγραφή (ξυπνά από `notify_all()` στο `process`)
> - Ή να λήξει το timeout (25 δευτ.) → επιστρέφει `204`
>
> **Αποτέλεσμα:** Μηδενική καθυστέρηση όταν έρχεται νέα περιγραφή, χωρίς συνεχείς αιτήσεις από την Flutter.

```
Flutter:  ──GET /caption_next?since=12345──▶  Server (κρατά ανοιχτό)
                                                     │
Pi:       ──POST /process──────────────────▶  Server (επεξεργάζεται)
                                                     │ (notify_all)
Flutter:  ◀──────────────{caption: "..."}────  Server (ξυπνά και απαντά)
```

🔧 `api/routers/tools.py` — Βοηθητικά Endpoints

> Αυτά τα endpoints είναι για **on-demand** χρήση (π.χ. testing, εξωτερικά agents). Δεν χρησιμοποιούνται στη βασική ροή Pi → Server → Flutter.

`GET /tools/vlm` και `POST /tools/vlm`

> Τρέχει αμέσως την περιγραφή στην **τελευταία αποθηκευμένη εικόνα** και επιστρέφει αποτέλεσμα. `POST` δέχεται JSON body με `image_path` (προαιρετικό).

`GET /tools/yolo` και `POST /tools/yolo`

> Τρέχει ανίχνευση YOLO στην τελευταία αποθηκευμένη εικόνα. Επιστρέφει αντικείμενα που ανιχνεύτηκαν + κατάσταση φαναριού (αν υπάρχει).

🔄 `pipelines/describe.py` — Η Καρδιά: Εικόνα → Περιγραφή

> Αυτό το αρχείο υλοποιεί την **pipeline** (αλυσίδα επεξεργασίας): παίρνει μια εικόνα, την προεπεξεργάζεται, την στέλνει στο μοντέλο AI και επιστρέφει κείμενο.

```
from v2.app.config import MODEL, PREPROC, DATA_DIR, PIPELINE_NAME,
get_device, get_dtype
```

> Εισάγει όλες τις ρυθμίσεις από το config.

```
def _load_prompt(path: str | None = None) -> str:
    """Load system prompt from file or return default."""
    ...
```

> Φορτώνει το **system prompt** — τις οδηγίες που δίνονται στο μοντέλο AI για το πώς να περιγράφει εικόνες. Διαβάζει από αρχείο στο `Data/system_prompts_qwen/`.

```
def _get_model_runner(model_name: str):
    """Return the appropriate model runner based on MODEL config."""
    if model_name == "qwen3_vl":
        from v2.app.models.Qwen.qwen3_vl_lmstudio import describe_image
        return describe_image
    elif model_name == "blip2":
        from v2.app.models.BLIP2... import describe_image
        return describe_image
    # ... κ.ο.κ.
```

> **Factory function:** Ανάλογα με την τιμή `MODEL` στο config, επιστρέφει τη σωστή συνάρτηση περιγραφής. Αυτό επιτρέπει την εύκολη εναλλαγή μοντέλων χωρίς αλλαγή του υπόλοιπου κώδικα.

```
def run(image_path: str, save: bool = True, profile: dict = None) -> str:
```

> Η κύρια συνάρτηση του pipeline. Καλείται από το `mobile.py` και τα `tools.py`.

Βήμα 1: Προεπεξεργασία εικόνας

```
    if PREPROC == "opencv":
        img_bgr = cv2.imread(image_path)
        img_rgb = preprocess_opencv(img_bgr)
        # Αποθήκευση ως temp file για το μοντέλο
    elif PREPROC == "pillow":
        img_pil = Image.open(image_path)
        img_pil = preprocess_pillow(img_pil)
```

> Χρησιμοποιεί `image_preprocess.py` για να βελτιώσει την ποιότητα της εικόνας πριν τη στείλει στο AI.

Βήμα 2: YOLO (αν ενεργό)

```
    if YOLO_ENABLED:
        from v2.app.pipelines.detect import run as run_detect
        yolo_result = run_detect(image_path)
        # Προσθήκη YOLO context στο prompt
```

> Αν το YOLO είναι ενεργό, εκτελεί ανίχνευση και **ενσωματώνει** τα αποτελέσματα στο prompt για το VLM.

Βήμα 3: Κλήση VLM

```
    describe_fn = _get_model_runner(MODEL)
    caption = describe_fn(image_path_or_pil, prompt=prompt)
```

> Καλεί το μοντέλο (π.χ. Qwen) για να πάρει την περιγραφή.

Βήμα 4: Αποθήκευση (αν save=True)

```
    if save:
        output_file = Path(DATA_DIR) / "Generated_Text" /
"image_description.txt"
        output_file.write_text(caption)
```

> Αποθηκεύει την περιγραφή στο δίσκο (κυρίως για debugging).

🔗 Συνδέσεις `describe.py`

Καλείται από	Καλεί
`mobile.py` (POST /process)	`models/Qwen/qwen3_vl_lmstudio.py`

Καλείται από	Καλεί
`tools.py` (GET/POST /tools/vlm)	`utils/image_preprocess.py`
	`pipelines/detect.py` (αν YOLO_ENABLED)

🎯 `pipelines/detect.py` — YOLO Pipeline

> Αυτό το αρχείο τρέχει ανίχνευση αντικειμένων με YOLO. Χρησιμοποιείται:
>
> 1. Μέσα στο `describe.py` (αν `YOLO_ENABLED=True`) για να εμπλουτίσει το context
> 2. Απευθείας από το `tools.py` (`/tools/yolo`)

```
def run(image_path: str, task: str = "cross_street") -> dict:
    from v2.app.models.yolo_model import detect_traffic_lights_with_color
    result = detect_traffic_lights_with_color(image_path)
    return result
```

> Καλεί το `yolo_model.py` για ανίχνευση φαναριών και εκτίμηση χρώματος.

🤖 `models/Qwen/qwen3_vl_lmstudio.py` — Επικοινωνία με LM Studio

> Αυτό το αρχείο μιλά στο **LM Studio** (το τοπικό πρόγραμμα που τρέχει το Qwen μοντέλο).

```
from openai import OpenAI
from v2.app.config import LM_STUDIO_BASE_URL, LM_STUDIO_MODEL_NAME
```

> - Χρησιμοποιεί τη βιβλιοθήκη `openai` — αλλά **δεν** μιλά στο ChatGPT! Το LM Studio εκθέτει **OpenAI-compatible API**, οπότε η ίδια βιβλιοθήκη δουλεύει και τοπικά.
> - Παίρνει URL και model name από config

```
client = OpenAI(
    base_url=LM_STUDIO_BASE_URL,  # http://127.0.0.1:9094
    api_key="not-needed"           # Δεν χρειάζεται πραγματικό κλειδί για
τοπικό server
)
```

> Δημιουργεί πελάτη (client) για επικοινωνία με το LM Studio.

```
def describe_image(image_path: str, prompt: str = None) -> str:
    # Κωδικοποίηση εικόνας σε base64
    with open(image_path, "rb") as f:
        image_data = base64.b64encode(f.read()).decode("utf-8")

    # Αποστολή στο Qwen
    response = client.chat.completions.create(
        model=LM_STUDIO_MODEL_NAME,
        messages=[
            {"role": "system", "content": prompt or DEFAULT_PROMPT},
            {"role": "user", "content": [
                {"type": "image_url", "image_url": {"url":
f"{image_data}"}},
                {"type": "text", "text": "Describe this image."}
            ]}
        ],
        max_tokens=512
    )
    return response.choices[0].message.content
```

> 1. Διαβάζει την εικόνα και τη μετατρέπει σε **base64** (κωδικοποίηση για αποστολή μέσω JSON)
> 2. Στέλνει αίτηση στο LM Studio με εικόνα + prompt
> 3. Επιστρέφει την περιγραφή ως string

🎨 `models/yolo_model.py` — YOLO: Ανίχνευση & Χρώμα Φαναριού

> Αυτό το αρχείο χρησιμοποιεί το **YOLO** (You Only Look Once) — ένα μοντέλο ανίχνευσης αντικειμένων σε πραγματικό χρόνο.

Φόρτωση Μοντέλου (Lazy Loading)

```
_MODEL: YOLO | None = None

def load_model(model_name: str | None = None) -> YOLO:
    global _MODEL
    if _MODEL is None:
        weights = YOLO_WEIGHTS
        _MODEL = YOLO(weights)
    return _MODEL
```

> - Το μοντέλο φορτώνεται **μόνο μια φορά** (lazy loading)
> - `global _MODEL` → ο μεταβλητός αυτός διατηρείται σε επίπεδο module (δεν ξαναφορτώνεται)
> - Αποτρέπει καθυστέρηση σε κάθε κλήση

Ανίχνευση Αντικειμένων

```
def detect_objects(image_path: str, ...) -> List[Dict]:
    mdl = load_model()
    img_bgr = _read_bgr_imdecode(image_path)
    results = mdl.predict(img_bgr, conf=0.25, save=False, ...)
    # Επιστρέφει: [{class, class_id, conf, bbox:[x1,y1,x2,y2]}, ...]
```

> - `conf=0.25` → κατώφλι εμπιστοσύνης: ανίχνευση μόνο αν το μοντέλο είναι >25% σίγουρο
> - `save=False` → δεν αποθηκεύει εικόνες αποτελεσμάτων

Εκτίμηση Χρώματος Φαναριού (HSV Analysis)

```
def infer_pedestrian_signal_state(image_path, bbox) -> Dict:
    # Κόβει το bbox από την εικόνα (inner crop 70%)
    # Μετατρέπει σε HSV χρωματικό χώρο
    # Μετράει κόκκινα και πράσινα pixels
    # Αποφασίζει: "red" | "green" | "unknown"
```

> **Πώς "βλέπει" χρώμα ένα πρόγραμμα;**
>
> - Κόβει το τμήμα της εικόνας όπου ανιχνεύτηκε φανάρι
> - Μετατρέπει σε **HSV** (Hue-Saturation-Value) — πιο κατάλληλο για ανίχνευση χρώματος από RGB
> - Μετράει πόσα pixels είναι στο εύρος χρώματος "κόκκινο" vs "πράσινο"
> - Αποφασίζει βάσει πλειοψηφίας

Πλήρης Ανίχνευση Φαναριών

```
def detect_traffic_lights_with_color(image_path) -> Dict:
    # 1. Ανίχνευση όλων των αντικειμένων
    # 2. Φιλτράρισμα: κράτα μόνο class_id=9 (traffic light)
    # 3. Φιλτράρισμα κατά μέγεθος (>0.5% του frame)
    # 4. Κράτα τα 2 μεγαλύτερα
    # 5. Για καθένα: εκτίμησε χρώμα
    # Επιστρέφει: {detections, pedestrian_signals, pedestrian_signal,
summary_color}
```

> **Αποτέλεσμα π.χ.:**
>
> ```
> {
>  "summary_color": "green",
>  "pedestrian_signal": {
> ```

```
      "state": "green",
      "confidence": 0.87,
      "position": "center"
    }
  }
```

🖼️ `utils/image_preprocess.py` — Βελτίωση Ποιότητας Εικόνας

> Πριν στείλει ο server εικόνα στο AI, την "βελτιώνει" με αυτές τις τεχνικές:

`preprocess_opencv` (default)

```
1. Noise reduction (NlMeans)   → αφαίρεση θορύβου/κόκκων
2. Upscale (αν <1024px)        → μεγέθυνση αν η εικόνα είναι μικρή
3. CLAHE contrast              → βελτίωση αντίθεσης (χρήσιμο σε σκοτεινές
σκηνές)
4. Unsharp mask                → ακόνισμα (sharpening)
5. BGR → RGB conversion        → αλλαγή χρωματικού χώρου (OpenCV
χρησιμοποιεί BGR)
```

`preprocess_pillow`

```
1. Upscale (αν <1024px)        → μεγέθυνση
2. UnsharpMask filter          → ακόνισμα
```

> **Γιατί χρειάζεται;** Οι εικόνες από κάμερα Pi μπορεί να είναι μικρές, θολές ή σκοτεινές. Η προεπεξεργασία βελτιώνει τη "λεπτομέρεια" που βλέπει το AI και άρα την ποιότητα της περιγραφής.

📱 `flutter_app/main.dart` — Η Εφαρμογή Κινητού

> Η Flutter εφαρμογή είναι ο **τελικός αποδέκτης** των περιγραφών. Τρέχει στο κινητό του χρήστη (π.χ. iPhone ή Android).

Κύρια Λειτουργία: Long-Polling Loop

```
Future<void> _pollCaption() async {
  while (mounted) {
    try {
      final url = Uri.parse(
        '$baseUrl/mobile/caption_next?
source=pi&since=$_lastCaptionTs&timeout=25'
      );
```

```
      final response = await http.get(url).timeout(Duration(seconds: 30));

      if (response.statusCode == 200) {
        final data = jsonDecode(response.body);
        setState(() {
          _caption = data['caption'];
          _lastCaptionTs = data['created_at'];
        });
        // Αυτόματη ανάγνωση μέσω TTS
        _tts.speak(_caption);
      }
      // 204 = δεν υπάρχει νέο → επανεκκίνηση αμέσως
    } catch (e) {
      await Future.delayed(Duration(seconds: 3)); // αναμονή αν υπάρχει
σφάλμα
    }
  }
}
```

> - `mounted` → η σελίδα είναι ακόμα ανοιχτή
> - `_lastCaptionTs` → timestamp της τελευταίας περιγραφής που είδε η εφαρμογή
> - `timeout=25` → ο server κρατά ανοιχτή τη σύνδεση 25 δευτ. αν δεν υπάρχει νέο
> - `Duration(seconds: 30)` → το κινητό κάνει timeout αν ο server δεν απαντήσει σε 30 δευτ.
> - `_tts.speak()` → Text-To-Speech: εκφωνεί την περιγραφή

Pi Status Check

```
Future<void> _pollPiStatus() async {
  // Καλεί /api/pi/status_next?since=...&timeout=15
  // Ανανεώνει το indicator (πράσινο/κόκκινο) στη UI
}
```

UI

> - Εμφανίζει την τελευταία περιγραφή ως κείμενο
> - Δείχνει αν το Pi είναι online
> - Κουμπί για manual refresh
> - Text-To-Speech εκφώνηση κάθε νέας περιγραφής

🔄 Πλήρης Ροή Λειτουργίας (End-to-End)

```
ΒΗΜΑ 1: Το Pi τραβά φωτογραφία
         ↓
ΒΗΜΑ 2: POST /mobile/process?src=pi
         ↓
```

```
ΒΗΜΑ 3: [mobile.py] Αποθήκευση εικόνας →
Data/received_images/current_image.jpg
         ↓
ΒΗΜΑ 4: [describe.py] Προεπεξεργασία εικόνας (OpenCV)
         ↓
ΒΗΜΑ 5: [describe.py] (Προαιρετικά) YOLO ανίχνευση φαναριού
         ↓
ΒΗΜΑ 6: [qwen3_vl_lmstudio.py] Κλήση LM Studio (Qwen VLM)
         • Εικόνα (base64) + System Prompt → HTTP POST → LM Studio
         • LM Studio (port 9094) επεξεργάζεται και επιστρέφει κείμενο
         ↓
ΒΗΜΑ 7: [mobile.py] Ενημέρωση state.latest_by_source["pi"]
         • state.last_seen_pi = τώρα
         • state.last_caption_ts["pi"] = τώρα
         ↓
ΒΗΜΑ 8: [mobile.py] notify_all() → ξυπνά η Flutter που περιμένει
         ↓
ΒΗΜΑ 9: [Flutter] /mobile/caption_next ξυπνά, παίρνει περιγραφή
         ↓
ΒΗΜΑ 10: [Flutter] Εμφανίζει κείμενο + TTS εκφώνηση
```

📊 Πλήρης Λίστα API Endpoints

Method	URL	Περιγραφή	Χρησιμοποιείται από
GET	`/`	Info endpoint	—
GET	`/health`	Health check	Monitoring
GET	`/api/pi/status`	Κατάσταση Pi (online/offline)	Flutter
GET	`/api/pi/status_next`	Long-polling Pi status	Flutter
POST	`/mobile/process`	**Ανεβάζει εικόνα** + επεξεργασία	**Raspberry Pi**
GET	`/mobile/caption_latest`	Τελευταία περιγραφή (άμεσα)	Flutter (fallback)
GET	`/mobile/caption_next`	**Long-polling** νέα περιγραφή	**Flutter**
GET	`/tools/vlm`	On-demand VLM στη τελευταία εικόνα	Testing/Agents
POST	`/tools/vlm`	On-demand VLM με custom path	Testing/Agents
GET	`/tools/yolo`	On-demand YOLO ανίχνευση	Testing/Agents
POST	`/tools/yolo`	On-demand YOLO με custom path	Testing/Agents

📦 Αρχεία Εξόδου

Αρχείο	Περιεχόμενο	Ενημερώνεται
`Data/received_images/current_image.jpg`	Τελευταία εικόνα από Pi/κινητό	Κάθε POST /process
`Data/Generated_Text/image_description.txt`	Τελευταία περιγραφή (plaintext)	Κάθε POST /process
`Data/Generated_Text/captions.json`	3 τελευταίες περιγραφές (JSON)	Κάθε POST /process
`Data/logs/timestamps.csv`	Χρόνοι επεξεργασίας	Κάθε POST /process (αν TIME_LOGS=ON)

🔧 Environment Variables (Μεταβλητές Περιβάλλοντος)

> Αυτές οι ρυθμίσεις μπορούν να αλλαχθούν **χωρίς τροποποίηση κώδικα**, απλώς ορίζοντάς τες στο σύστημα:

Μεταβλητή	Default	Περιγραφή
`DATA_DIR`	`VLM_v2/Data`	Φάκελος δεδομένων
`LM_STUDIO_BASE_URL`	`http://127.0.0.1:9094`	URL του LM Studio
`LM_STUDIO_MODEL_NAME`	`qwen3-vl-32b-instruct`	Όνομα μοντέλου στο LM Studio
`YOLO_WEIGHTS`	`models/weights/yolo26n.pt`	Path στα βάρη YOLO
`TIME_LOGS`	`ON`	Ενεργοποίηση log χρόνων (ON/OFF)
`YOLO_ENABLED`	`OFF`	Ενεργοποίηση YOLO pipeline (ON/OFF)

💡 Σύνοψη Κλειδιών Αρχιτεκτονικής

Έννοια	Επεξήγηση
FastAPI	Web framework για γρήγορα async HTTP APIs
Uvicorn	ASGI server που τρέχει το FastAPI
Long-Polling	Η Flutter ανοίγει σύνδεση, ο server την κρατά "σε αναμονή" μέχρι να υπάρξει νέο δεδομένο
asyncio.Condition	Το "καμπανάκι" που ξυπνά waiters όταν αλλάξει κάτι
State (RAM)	Δεδομένα που ζουν μόνο κατά τη λειτουργία του server

Έννοια	Επεξήγηση
Thread Pool	Βαριές λειτουργίες (LM Studio) τρέχουν σε ξεχωριστό thread για να μην "παγώσει" ο server
YOLO	Ανίχνευση αντικειμένων σε πραγματικό χρόνο (φανάρια)
VLM	Vision Language Model — AI που "βλέπει" και "μιλά" (Qwen)
LM Studio	Τοπικό πρόγραμμα που τρέχει μεγάλα AI μοντέλα
Base64	Κωδικοποίηση binary δεδομένων (εικόνα) σε κείμενο για αποστολή μέσω JSON
HSV	Χρωματικός χώρος κατάλληλος για ανίχνευση χρωμάτων
TTS	Text-To-Speech: εκφώνηση κειμένου (Flutter → χρήστης)

Τέλος τεκμηρίωσης — VLM_v2 Gateway Server